

Singleton Design Pattern

Intent

Guarantee that a class has **exactly one instance** for the whole lifetime of the JVM, and give everyone a single global access point to reach it.

This implementation is the **thread-safe, lazily-initialized** variant, built with the **double-checked locking** idiom over a private `volatile` field and a private mutex object.

UML class diagram

```
+-----+
| SingletonPattern |
+-----+
| - instance : SingletonPattern {static,volatile}
| - mutex : Object {static,final}
+-----+
| - SingletonPattern() <<private ctor>>
| + getInstance() : SingletonPattern {static}
| + msg() : void
+-----+
client --> SingletonPattern.getInstance() --> the ONE shared instance
```

The code, line by line

```
package com.design.patterns.singleton;

public class SingletonPattern {

    private static volatile SingletonPattern instance;
    private static final Object mutex = new Object();

    private SingletonPattern() {
    }

    public static SingletonPattern getInstance() {
        SingletonPattern result = instance;
        if (result == null) {
            synchronized (mutex) {
                result = instance;
                if (result == null) {
                    instance = result = new SingletonPattern();
                }
            }
        }
        return result;
    }

    public void msg() {
        System.out.println("Threadsafe singleton design pattern implemented");
    }
}
```

`private static volatile SingletonPattern instance;`

- `static` — the single instance is held on the **class**, not on any object. There is one slot per JVM, which is exactly what "single instance" means.
- `private` — nobody outside the class can touch the field directly; the only way in is through `getInstance()`. This is what lets the class **enforce** the guarantee instead of just hoping callers behave.
- `volatile` — this is the critical one for correctness. Explained in depth in [\[Why volatile ?\]\(#why-volatile\)](#) below.
- It is **not** initialized here (no `= new SingletonPattern()`). That makes the initialization **lazy** — the object is only built the first time someone actually asks for it, not when the class loads.

`private static final Object mutex = new Object();`

- A dedicated lock object. `getInstance()` synchronizes on **this** object rather than on `SingletonPattern.class` or on `this`.
- `final` — the lock reference never changes, so every thread locks on the **same** monitor. A lock you could reassign would be a broken lock.
- Explained in depth in [\[Why a private mutex?\]\(#why-a-private-mutex-and-why-my-own-one\)](#) below.

`private SingletonPattern() { }`

- The constructor is **private**. This is the wall that makes the pattern real: no other class can write `new SingletonPattern()`. If the constructor were public, callers could make as many instances as they liked and the "single" guarantee would be a lie.
- The only code allowed to call this constructor is `getInstance()`, inside this same class.

`getInstance()` — the double-checked locking flow

```
SingletonPattern result = instance; // (1) read the volatile ONCE into a local
if (result == null) {               // (2) first check – no lock
    synchronized (mutex) {         // (3) only contend for the lock if it looks unbuilt
        result = instance;         // (4) re-read inside the lock
        if (result == null) {      // (5) second check – now under mutual exclusion
            instance = result = new SingletonPattern(); // (6) build exactly once
        }
    }
}
return result;                      // (7) hand back the one instance
```

1. **`SingletonPattern result = instance;`** — copy the `volatile` field into a local variable. This is a deliberate optimization: reading a `volatile` is more expensive than reading a local, and on the common path (instance already exists) we want to touch the volatile only once. This is the *"local variable"* micro-optimization popularized by Effective Java.
2. **`First if (result == null)`** — the fast path. Once the singleton has been created, `result` is non-null and we skip the `synchronized` block **entirely**. This means after the one-time setup, `getInstance()` costs nothing more than one volatile read — no locking, no contention.
3. **`synchronized (mutex)`** — we only reach here when it *"looks"* like the instance hasn't been built yet. We take the lock so that only one thread at a time can proceed to create it.
4. **`result = instance;`** — re-read the field now that we hold the lock. Another thread may have created the instance between our first check (step 2) and us acquiring the lock (step 3).
5. **`Second if (result == null)`** — the "double check". If some other thread already built the instance while we were waiting for the lock, this is now non-null and we do **not** build a second one.
6. **`instance = result = new SingletonPattern();`** — the one and only construction. Assigns to the local (`result`) and publishes to the `volatile` field (`instance`) in one statement.
7. **`return result;`** — every caller, first or millionth, receives the identical object.

`msg()`

An ordinary instance method, just here to prove the returned object is usable: `SingletonPattern.getInstance().msg()` prints the confirmation line.

Why the design decisions (the "why", not the "what")

Why lazy initialization at all?

The instance is built on first use rather than at class-load time. Useful when constructing the singleton is expensive (opens connections, reads files, allocates big buffers) and you don't want to pay that cost — or risk that failure — unless something actually needs it. The trade-off is that lazy creation is exactly what forces us to think about threads, which is where everything below comes from.

Why worry about threads?

If two threads call `getInstance()` at the same time on a naive lazy singleton:

```
if (instance == null) {             // both threads see null
    instance = new SingletonPattern(); // both construct → TWO instances
}
```

Both can pass the `null` check before either assigns, and you end up with **two** instances. That breaks the entire guarantee. So a lazy singleton *must* be synchronized somehow.

Why not just `synchronized` the whole method?

The simplest fix is:

```
public static synchronized SingletonPattern getInstance() {
    if (instance == null) instance = new SingletonPattern();
    return instance;
}
```

This is correct but **slow**: **every** call acquires the lock forever, even though the lock is only ever needed for the single instant of creation. If the singleton is read on a hot path by many threads, they all serialize on that lock for no reason. Double-checked locking exists to pay the lock cost **only once** — during creation — and run lock-free afterwards.

Why a private mutex, and why "my own" one?

Instead of `synchronized(SingletonPattern.class)` or `synchronized(this)`, this code locks on a **private Object mutex** that it owns. The reason is **encapsulation of the lock**:

- If you lock on `SingletonPattern.class`, that monitor is **publicly reachable** — any other code anywhere can also write `synchronized(SingletonPattern.class) { ... }` and now they are contending with, or even deadlocking, your singleton's internal locking. You don't control who else grabs a public lock.
- `synchronized(this)` isn't even an option here because creation happens in a `static` method (there is no `this` yet), but the same argument applies to instance-level singletons: `this` leaks to callers who could lock on it.
- A **private final Object mutex** is invisible outside the class. Nobody else can acquire it, so the lock's behavior is entirely under this class's control. This is the standard "private lock object" idiom — the lock is an implementation detail, not part of the public API.

`final` matters too: it means the mutex reference can never be reassigned, so all threads always synchronize on the **same** monitor. A non-final lock is a classic concurrency bug.

Why volatile?

This is the subtle heart of double-checked locking. Without `volatile`, DCL is **broken** — and it was famously broken in Java before the memory model was clarified.

The dangerous line is:

```
instance = new SingletonPattern();
```

This is **not** atomic. It is really three steps:

1. **allocate** memory for the object,
2. **run the constructor** to initialize the fields,
3. **publish** — assign the reference to `instance`.

The JVM/CPU is allowed to **reorder** steps 2 and 3 when there is no `volatile`. If publication (3) happens **before** construction (2) finishes, then:

- Thread A is mid-construction: `instance` already points at a **half-built** object.
- Thread B calls `getInstance()`, sees `instance != null` on the fast path (step 2 of the flow), skips the lock, and **returns the half-constructed object**. Thread B then uses an object whose fields aren't initialized yet → corrupt state, random crashes.

`volatile` fixes this in two ways under the Java Memory Model (Java 5+):

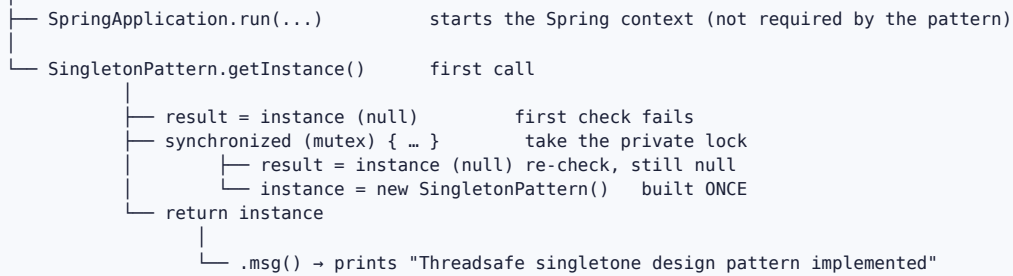
1. It **forbids that reordering** — the write to a `volatile` field cannot be moved before the writes that happen-before it, so the constructor is guaranteed to be fully done before the reference is visible.
2. It establishes a **happens-before / visibility** relationship: once thread A writes the `volatile`, any thread that later reads it sees **all** the writes A made before it (the fully initialized fields), not a stale cached copy.

Without `volatile`, a thread reading `instance` on the lock-free fast path could also just see a **stale null** (from its CPU cache) even after another thread created the instance, or worse, the half-built object above. `volatile` is what makes the lock-free first check actually safe.

In one sentence: the `synchronized` / mutex block prevents **two** instances from being **created**; `volatile` prevents **other threads** from **seeing** a partially-created one. You need both — that's why it's called double-checked **locking** over a **volatile** field.

Execution flow (as run from `main`)

DesignPatternsApplication.main



Every later getInstance() call: result = instance (non-null) → returns immediately, no lock.

Alternatives worth knowing (and why this one was chosen)

Approach	One instance?	Thread-safe?	Lazy?	Lock cost
Eager static final field	yes	yes (class-init is safe)	no	none
synchronized whole method	yes	yes	yes	every call
Double-checked locking (this code)	yes	yes	yes	once
Initialization-on-demand holder (static nested class)	yes	yes	yes	none
enum singleton	yes	yes	no	none

This module demonstrates **double-checked locking** specifically because it is the classic teaching example for **why** `volatile` and a dedicated lock are needed. In production, the **holder idiom** or an `enum` are often simpler and just as safe — but they hide the memory-model lesson that DCL makes explicit.

Note: `SpringApplication.run(...)` in `main` just boots a Spring context; it is unrelated to the pattern. The singleton here is managed manually (classic GoF), **not** by Spring's container. A Spring `@Component` / `@Bean` is already a singleton within its context, but that is a different, framework-managed mechanism.